

OPserver_3.0: user manual

C. Mendoza

Physics Center, IVIC, Caracas, Venezuela

`claudio@ivic.ve`

1 Introduction

OPserver is an extension of the suite of FORTRAN codes recently made available by Seaton [1, 2] for computing radiative opacities and accelerations using the Opacity Project (OP) data. The codes have been re-structured as a subroutine library, referred to as the OPlibrary, which can be linked by a user program to simplify and increase the efficiency of the opacity calculations. One of its attractive features is that OPserver calculates mean opacities and radiative accelerations with all the monochromatic opacities in main memory (~ 1 Gb) and then store them in a memory buffer, thus speeding further computations in stellar modelling where the chemical mixture changes frequently with depth and/or time. Two user modes are available:

1. A local version where the monochromatic opacities are installed locally (see file **README**).
2. A remote version where the monochromatic opacities are accessed from a remote site (the Ohio Supercomputer Center [3]) through the Internet.

The present manual gives brief instructions on how to install, link and run the OPlibrary offered by the OPserver contained in the OPCD_3.0 release. It requires a FORTRAN-90 compiler and at least 1 Gb of RAM for adequate running of the local version. The remote version is in practice not limited by RAM size.

2 Installation

2.1 Local version

The local version of OPserver requires beforehand the installation of the OP codes and data which, as specified in the file **README**, is performed with the commands

```
./configure (1)
```

```
make (2)
```

```
make install (3)
```

The **configure** procedure searches for the active FORTRAN-90 compiler and its corresponding compiler options which are then written in the file **Makefile.incl**. This file is then read by the **make** utility to complete installation. **Makefile.incl** can be edited by the installer

if in need of a different compiler choice or changes in the compiler options. OPserver is then installed in the OPCODE_3.0 directory with the command

```
make opserver (4)
```

The installation procedure compiles UNIX library files for both single precision

```
./opserver/libopserver_sp.a (5)
```

and double precision

```
./opserver/libopserver_dp.a (6)
```

and a set of FORTRAN-90 module files. Installation can be tested with the command

```
make test_opserver (7)
```

which compiles and runs the executable `optest.x`. This program reads the test input files in subdirectory `testin` and writes the files `opmix.s92`, `opfit.s92`, `mx.s92.g`, `acc.18`, `accfit.18` and `ax.18.g` which can then be compared with the test-runs in subdirectory `testout`. Some additional integers are written in the OPserver files which are unimportant, but differences in the floating point data should be small.

2.2 Remote version

The remote version of OPserver computes mean opacities and radiative accelerations accessing the monochromatic opacities from the Ohio Computer Center; thus it does not require the local installation of the OP data. However, it uses for Internet data transfers routines compiled with the GNU `gcc` compiler [4] and linkage with the `libstdc++` [4] and the `cURL` libraries [5]. Such software is now native in most UNIX compilers but can also be easily downloaded and installed. Local availability of `libcurl` be verified with the command

```
curl-config (8)
```

OPserver is installed with the commands

```
./configure (9)
```

```
make (10)
```

```
make opserver_OSC (11)
```

and tested with

```
make test_opserver_OSC (12)
```

which executes the same tasks as the local version (see Section 2.1).

3 Library Linking

Although OPserver always computes mean opacities and radiative accelerations in single precision, it provides interfaces for user codes in either precision. In the following sections, instructions are given in turn for each precision mode.

3.1 Local version

3.1.1 Single precision

The OPlibrary can be linked to a user code at compilation time with the following specifications

$$\begin{aligned} & \$(FC) \quad -omyprog.x \quad myprog.f \quad \$(MYINCLPATH) \quad \$(MYMODPATH) \\ & \qquad \qquad \qquad \$(MYLIBPATH) \quad -lopserver_sp \end{aligned} \tag{13}$$

where the variable $\$(FC)$, listed in file `Makefile.incl` for each installation, invokes the FORTRAN-90 compiler. *myprog.f* and *myprog.x* are the user FORTRAN program (single precision) and executable, respectively. The variables $\$(MYINCLPATH)$, $\$(MYMODPATH)$ and $\$(MYLIBPATH)$, also listed in `Makefile.incl`, respectively specify the `include`, `module` and `library` paths, the latter for the single-precision library `libopserver_sp.a`. For most compilers

$$\$(MYMODPATH) = \$(MYINCLPATH) \tag{14}$$

so $\$(MYMODPATH)$ needs not be specified at the linking step.

The OPserver module must be declared within the user-program unit where it is to be called with the FORTRAN-90 statement

$$\text{use opserver_sp} \tag{15}$$

(see, for instance, file `./opserver/optest.f`).

3.1.2 Double precision

In a similar fashion, the linking options in double precision are

$$\begin{aligned} & \$(FC) \quad -omyprog.x \quad myprog.f \quad \$(MYINCLPATH) \quad \$(MYMODPATH) \\ & \qquad \qquad \qquad \$(MYLIBPATH) \quad -lopserver_dp \end{aligned} \tag{16}$$

where *myprog.f* is a double-precision FORTRAN source, or alternatively

$$\begin{aligned} & \$(FC) \quad -r8 \quad -omyprog.x \quad myprog.f \quad \$(MYINCLPATH) \\ & \qquad \qquad \qquad \$(MYMODPATH) \quad \$(MYLIBPATH) \quad -lopserver_dp \end{aligned} \tag{17}$$

where now *myprog.f* is now a single-precision source compiled with the double precision option `r8`.

The FORTRAN-90 declaration

$$\text{use opserver_dp} \tag{18}$$

must be included within the user-program unit from where OPserver module is to be called.

3.2 Remote version

3.2.1 Single precision

The OPlibrary can be linked to a user code at compilation time with the following specifications

$$\begin{aligned} & \$(FC) \quad -omyprog.x \quad myprog.f \quad \$(MYINCLPATH_OSC) \quad \$(MYMODPATH_OSC) \\ & \quad \quad \quad \$(MYLIBPATH_OSC) \quad -loptserver_sp \quad \$(LIBCURL) \end{aligned} \quad (19)$$

which are similar to those of the local version (see Section 3.1.1) except for the extra libraries associated with the variable $\$(LIBCURL)$ (see file `Makefile.incl`).

3.2.2 Double precision

The linking options in double precision are

$$\begin{aligned} & \$(FC) \quad -omyprog.x \quad myprog.f \quad \$(MYINCLPATH_OSC) \quad \$(MYMODPATH_OSC) \\ & \quad \quad \quad \$(MYLIBPATH_OSC) \quad -loptserver_dp \quad \$(LIBCURL) \end{aligned} \quad (20)$$

for the double-precision FORTRAN source *myprog.f*, or alternatively, for a single-precision source

$$\begin{aligned} & \$(FC) \quad -r8 \quad -omyprog.x \quad myprog.f \quad \$(MYINCLPATH_OSC) \\ & \quad \quad \quad \$(MYMODPATH_OSC) \quad \$(MYLIBPATH_OSC) \quad -loptserver_dp \quad \$(LIBCURL) \end{aligned} \quad (21)$$

Again, the FORTRAN-90 statement

$$\text{use } optserver_dp \quad (22)$$

must be included to declare the OPserver module within the user-program unit from where OPserver is to be called.

4 Subroutines

The subroutines in the OPlibrary are essentially adaptations of Seaton's programs [1], the main difference being that mean opacities and radiative accelerations are computed with the monochromatic opacities loaded in RAM, and then stored in an internal memory buffer for further processing by the user code. Such computations are managed by subroutine calls which are described below. A flow chart is depicted in Fig. 1.

4.1 op_dload

This is the first subroutine to be called by the local version of OPserver. It loads the complete set of monochromatic opacity and acceleration files in an internal memory buffer where they

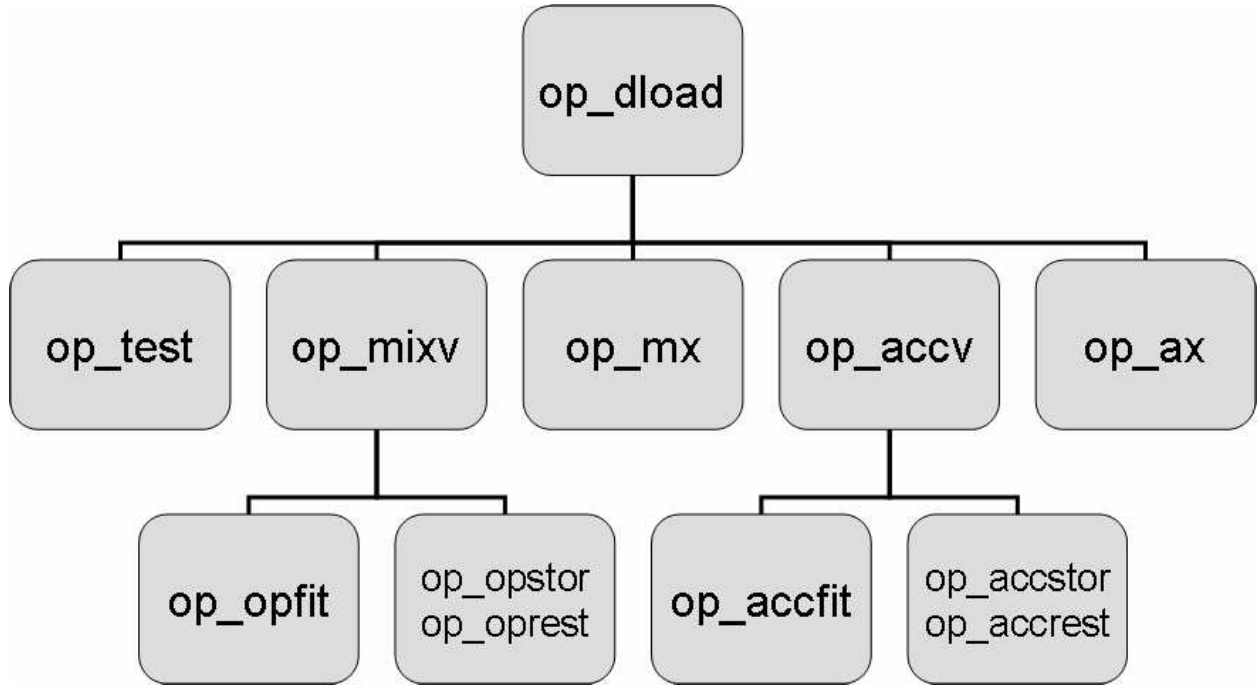


Figure 1: OPserver flow chart.

can be rapidly accessed for further computations, namely mean opacities with subroutine `op_mixv` or accelerations with `op_accv`.

```
subroutine op_dload(ierr) (23)
```

`op_dload` returns `ierr = 0` if data have been properly loaded.

Note: in the remote version of OPserver there is no need to call this routine.

4.2 op_test

After loading the monochromatic opacities in RAM with subroutine `op_dload` (local version), subroutine `op_test` can be used to test correct linking. It performs the same tasks as the program `optest.f` described above.

```
subroutine op_test(ierr) (24)
```

`op_test` returns `ierr = 0` for correct running.

Note: In the remote version, this routine can be invoked without calling `op_dload` beforehand.

4.3 op_mixv

Once the procedure `op_dload` has been invoked (local version), subroutine `op_mixv` is run to compute—for a user specified chemical mixture— Rosseland and Planck mean opacities

in the entire $\{\log T, \log \rho\}$ plane. This intermediate data set is kept in an internal memory buffer where it can be used for rapid and extensive interpolations with subroutine `op_opfit` or written to disk with `op_opstor`. The input chemical mixture is specified by its: (i) hydrogen mass fraction X ; metal mass fraction Z and (iii) the metal fractional abundances. The routine outputs the renormalized fractional abundances for the complete mixture.

subroutine `op_mixv(x,z,nel,iz,fa,ierr)` (25)

Input

`x` Hydrogen mass fraction (real)
`z` Metal mass fraction (real)
`nel` Number of elements in the mixture (integer)
`iz(1:nmel)` Nuclear charges Z for the `nmel` metals (integer array)
`fa(1:nmel)` Fractional abundances for the `nmel` metals (real array)

Output

`iz(1:nel)` Nuclear charges Z for the `nel` elements in the mixture (integer array)
`fa(1:nel)` Fractional abundances for the `nel` elements (real array)
`ierr` Returns 0 for correct use

Note: in the remote version, this routine can be invoked without the prior calling of `op_dload`.

4.4 `op_opstor`

This routine can be run after `op_mixv` to write to disk the intermediate opacity file that it computes. This file can be read back to RAM with subroutine `op_oprest` for further use by `op_opfit`. It returns `ierr` = 0 for correct download.

subroutine `op_opstor('filnam',ierr)` (26)

Input

`filnam` Name of file to be written to disk (character)

4.5 `op_oprest`

This routine reads from disk an intermediate opacity file placing it in an internal buffer for further processing by `op_opfit`. It returns `ierr` = 0 if the file is correctly read.

subroutine `op_oprest('filnam',ierr)` (27)

Input

filnam	Name of file to be read from disk (character)
---------------	---

4.6 op_opfit

After routine `op_mixv` has been run to compute mean opacities for a specific chemical mixture on the entire $\{\log T, \log \rho\}$ array, `op_opfit` is run to obtain fitted Rosseland means and derivatives for a tabulation of $\{\log T, \log \rho\}$ points.

$$\text{subroutine op_opfit}(\text{np}, \text{flt}, \text{flrho}, \text{g}, \text{dgdt}, \text{dgdrho}, \text{ierr}) \quad (28)$$

Input

np	Number of $\{\log T, \log \rho\}$ points (integer)
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	10
11	11
12	12
13	13
14	14
15	15
16	16
17	17
18	18
19	19
20	20
21	21
22	22
23	23
24	24
25	25
26	26
27	27
28	28
29	29
30	30
31	31
32	32
33	33
34	34
35	35
36	36
37	37
38	38
39	39
40	40
41	41
42	42
43	43
44	44
45	45
46	46
47	47
48	48
49	49
50	50
51	51
52	52
53	53
54	54
55	55
56	56
57	57
58	58
59	59
60	60
61	61
62	62
63	63
64	64
65	65
66	66
67	67
68	68
69	69
70	70
71	71
72	72
73	73
74	74
75	75
76	76
77	77
78	78
79	79
80	80
81	81
82	82
83	83
84	84
85	85
86	86
87	87
88	88
89	89
90	90
91	91
92	92
93	93
94	94
95	95
96	96
97	97
98	98
99	99
100	100

<code>flt(1:np)</code>	Point temperature $\log T$ (real array)
------------------------	---

flrho(1:np) Point density $\log \rho$ (real array)

Output

g(1:np)	Point Rosseland mean opacity $\log \kappa_R$ for point (real array)
---------	---

dgdt(1:np) Point temperature derivative $\partial \log \kappa_R / \partial \log T$ (real array)

dgdrho(1:np) Point density derivative $\partial \log \kappa_R / \partial \log \rho$ (real array)

ierr	Returns 0 for correct use (integer)
------	-------------------------------------

4.7 op_mx

After OPserver is invoked with subroutine `op_dload` (local version), Rosseland mean opacities and derivatives can be computed with subroutine `op_mx` for a multi-mixture depth profile of $\{\log T, \log \rho\}$ points. This routine is not available in the remote version of OPserver.

$$\text{subroutine op_mx(nel,nmix,np,iz,fa,flt,flrho,imix,} \\ \text{g,dgdt,dgdrho,ierr)} \quad (29)$$

Input

nel Number of elements in chemical mixtures (integer)

nmix	Number of chemical mixtures in the depth profile (integer)
-------------	--

`iz(1:nel,1:nmix)` Nuclear charge Z for the `nel` elements in each of the `nmix` mixtures
(integer array)

`fa(1:nel,1:nmix)` Fractional abundance for the `nel` elements in each of the `nmix` mixtures
(integer array)

np	Number of $\{\log T, \log \rho\}$ points in the depth profile (integer)
----	---

flt(1:np) Point temperature $\log T$ (real array)
flrho(1:np) Point density $\log \rho$ (real array)
imix(1:np) Point chemical mixture index ($1 \leq \text{imix} \leq \text{nmix}$) (integer array)

Output

g(1:np) Point Rosseland mean opacity $\log \kappa_R$ for point (real array)
dgdT(1:np) Point temperature derivative $\partial \log \kappa_R / \partial \log T$ (real array)
dgdrho(1:np) Point density derivative $\partial \log \kappa_R / \partial \log \rho$ (real array)
ierr Returns 0 for correct use (integer)

4.8 op_accv

After OPserver (local version) has been loaded with subroutine **op_dload**, radiative acceleration data are computed with subroutine **op_accv** in the entire $\{\log T, \log \rho\}$ plane for a selected element of a user-specified chemical mixture and a range of abundance multipliers χ . The computed data set is kept in an internal buffer where they can be interpolated with subroutine **op_accfit** for a stellar depth profile or written to disk with **op_accstor**. For correct running, **ierr** = 0 is returned.

$$\text{subroutine op_accv}(\text{iz1}, \text{nel}, \text{iz}, \text{fa}, \text{nchi}, \text{chil0}, \text{dchil}, \text{ierr}) \quad (30)$$

Input

iz1 Nuclear charge Z for selected element (integer)
nel Number of elements in chemical mixture (integer)
iz(1:nel) Nuclear charge Z for the **nel** elements in the mixture (integer array)
fa(1:nel) Fractional abundance for the **nel** elements in the mixture (real array)
nchi Number of abundance multipliers χ (integer)
chil0 Lowest value of the abundance multiplier ($\log \chi$) range (integer)
dchil Interval in the abundance multiplier ($\log \chi$) range (integer)

Note: In the remote version, this routine can be invoked without calling **op_dload** beforehand.

4.9 op_accstor

After subroutine **op_accv** has been called, the intermediate acceleration file that it computes for the entire $\{\log T, \log \rho\}$ array can be written to disk with subroutine **op_accstor**. The downloaded file may be read back to RAM with subroutine **op_accrest**. **op_accstor** returns **ierr** = 0 for correct downloading.

$$\text{subroutine op_accstor}(\text{'filnam'}, \text{ierr}) \quad (31)$$

Input

filnam Name of file to be written (character)

4.10 op_accrest

The intermediate acceleration file computed with subroutine **op_accv** and downloaded to disk with **op_accstor** can be read back to RAM with subroutine **op_accrest**. It is placed in an internal buffer for further processing with subroutine **op_accfit**. Subroutine **op_accrest** returns **ierr** = 0 for successful file reading.

subroutine op_accrest('filnam',ierr) (32)

Input

filnam Name of file to be read (character)

4.11 op_accfit

The acceleration file computed with subroutine **op_accv** is kept in an internal buffer for fast processing with subroutine **op_accfit**. The latter computes, for a range of abundance multipliers χ and a stellar model with effective temperature T_{eff} and depth profile $\{\log T, \log \rho, r/R_\star\}$, the following radiative acceleration data: ζ parameter used in the diffusion coefficient; the Rosseland mean $\log \kappa_R$; $\log \gamma$ and $\log g_{\text{rad}}$.

subroutine op_accfit(nchi,chil0,dchil,teff,nstar,flt,flrho,ri,
 zet,rss,gmm,grd,ierr) (33)

Input

nchi Number of abundance multipliers χ (integer)
chil0 Lowest value of the abundance multiplier ($\log \chi$) range (real)
dchil Interval in the abundance multiplier ($\log \chi$) range (real)
teff Effective temperature T_{eff} of stellar model (real)
nstar Number of $\{\log T, \log \rho, r/R_\star\}$ depth points (integer)
flt(1:nstar) Point temperature $\log T$ (real array)
flrho(1:nstar) Point density $\log \rho$ (real array)
ri(1:nstar) Point fractional depth $r/R_\star$ (real array)

Output

zet(1:nstar,1:nchi) ζ for **nchi** values of χ (real array)
rss(1:nstar,1:nchi) Rosseland mean opacity $\log \kappa_R$ for **nchi** values of χ (real array)
gmm(1:nstar,1:nchi) $\log \gamma$ for **nchi** values of χ (real array)

`grd(1:nstar,1:nchi)` $\log g_{\text{rad}}$ for `nchi` values of χ (real array)

`ierr` Returns 0 for correct use (integer)

4.12 op_ax

After the OPserver (local version) has been loaded with subroutine `op_dload`, subroutine `op_ax` can be run to compute acceleration data for a multi-mixture stellar model. For a selected chemical element, a range of χ abundance multipliers and a star model with effective temperature T_{eff} and depth profile $\{\log T, \log \rho, r/R_\star, i_{\text{mix}}\}$, where $i_{\text{mix}} = 1, \text{nmix}$ is the mixture index, subroutine `op_ax` outputs: ζ ; the Rosseland mean $\log \kappa_R$; $\partial \log \kappa_R / \partial \log \chi$; $\log \gamma$; $\partial \log \gamma / \partial \log \chi$ and $\log g_{\text{rad}}$. This routine is not available in the remote version of OPserver.

```
subroutine    op_ax(iz1,nel,nmix,nch,nstar,iz,fa,chil0,dchil,
               teff,flt,flrho,ri,imix,zet,g,gp,f,fp,grl,ierr)      (34)
```

Input

`iz1` Nuclear charge Z for selected element (integer)

`nel` Number of elements in chemical mixture (integer)

`nmix` Number of chemical mixtures in the depth profile (integer)

`nch` Number of abundance multipliers χ (integer)

`nstar` Number of $\{\log T, \log \rho, r/R_\star\}$ plasma points (integer)

`iz(1:nel,1:nmix)` Nuclear charge Z for the `nel` elements in each of the `nmix` mixtures (integer array)

`fa(1:nel,1:nmix)` Fractional abundance for the `nel` elements in each of the `nmix` mixtures (integer array)

`chil0` Lowest value of the abundance multiplier ($\log \chi$) range (real)

`dchil` Interval in the abundance multiplier ($\log \chi$) range (real)

`teff` Effective temperature T_{eff} of stellar model (real)

`flt(1:nstar)` Point temperature $\log(T)$ (real array)

`flrho(1:nstar)` Point density $\log(\rho)$ (real array)

`ri(1:nstar)` Point fractional depth $r/R_\star$ (real array)

`imix(1:nstar)` Point chemical mixture index ($1 \leq \text{imix} \leq \text{nmix}$) (integer array)

Output

`zet(1:nstar,1:nch)` ζ parameter used for diffusion coefficient (real array)

`g(1:nstar,1:nch)` Rosseland mean opacity $\log \kappa_R$ (real array)

`gp(1:nstar,1:nch)` Derivative $\partial \log \kappa_R / \partial \log \chi$ (real array)

`f(1:nstar,1:nch)` $\log \gamma$ (real array)
`fp(1:nstar,1:nch)` $\partial \log \gamma / \partial \log \chi$ (real array)
`grl(1:nstar,1:nch)` $\log g_{\text{rad}}$ (real array)
`ierr` Returns 0 for correct use (integer)

References

1. Seaton, M.J. 2005, MNRAS, 362, L1
2. <http://www.osc.edu/hpc/opacities/OPCD.pdf>
3. <http://www.osc.edu/hpc/opacities/>
4. <http://gcc.gnu.org/>
5. <http://curl.haxx.se/>